Ratatoskr

Leoluca Bernardi, Leonardo Bolognese, Ali Gorgani, Mattia Metzler, Anthony Tamberg

Contents

Ratatoskr . . . 1
1. Abstract . . . 2
2. The Maze . . . 2
   2.1. Technical Drawing . . . 2
3. The Mouse . . . 3
   3.1. Design and Technical Drawing . . . 3
   3.2. Hardware . . . 4
      3.2.1. Motors, Drivers & Encoders . . . 4
      3.2.2. Gyroscope . . . 4
      3.2.3. TOF sensors . . . 5
   3.3. Software . . . 5
      3.3.1. Hardware Interfaces . . . 5
      3.3.2. Movement Control . . . 8
      3.3.3. PID Controllers (Proportional–integral–derivative) . . . 8
      3.3.4. Maze Solver . . . 9
      3.3.5. LED Matrix & SD Card . . . 9
4. Tasks and Milestones . . . 10
   4.1. Ratatoskr V1.0 . . . 10
   4.2. Ratatoskr V1.1 . . . 11
   4.3. Ratatoskr V1.2 . . . 11
   4.4. Ratatoskr Final . . . 11
5. User Stories . . . 11
6. Risk Assessment . . . 12
   6.1. Hardware Risks . . . 12
   6.2. Software Risks . . . 12
   6.3. Human Resource Risks . . . 13
7. Bill of Materials . . . 14
8. Inspiration and References . . . 14

1. Abstract

Ratatoskr is a small autonomous wheeled robot, a 'micromouse', that can find the fastest path through a maze. In two separate `search` and `run` phases, the mouse finds the most optimal path and then traverses it, accomplishing both in the shortest possible time. This project's aim is to improve the design and software of the previous year's micromice, as well as the design of the maze that they solve. The final result will be a faster micromouse wandering a smoother maze.

2. The Maze

Our maze for Ratatoskr will be made up of cells measuring 18 cm per side. Previous designs used modular walls made from 16cm planks attached to posts, but the posts were held in place with Velcro instead of screws. This caused misalignment and uneven walls, which negatively affected the performance of the micromouse.

Our new design (see Figure 1) reuses the wall pieces from last year, but using their intended screw-in attachment. Each floor tile covers a 36×36 cm^2 area, not including the connecting attachments. One such tile fits four maze cells. The posts will be screwed directly into the tiles, keeping the structure rigid and aligned. The thickness of the tiles will be between 3-5 mm and with countersunk screw holes, these tiles will sit flush on the ground.

The tiles have been designed so that seams in the floor only run perpendicular to any path taken through the maze, so that the wheels of the micromouse cannot be caught in between two tiles.

2.1. Technical Drawing

Below is a technical drawing of the tile, showing it's exact dimensions in millimeters.

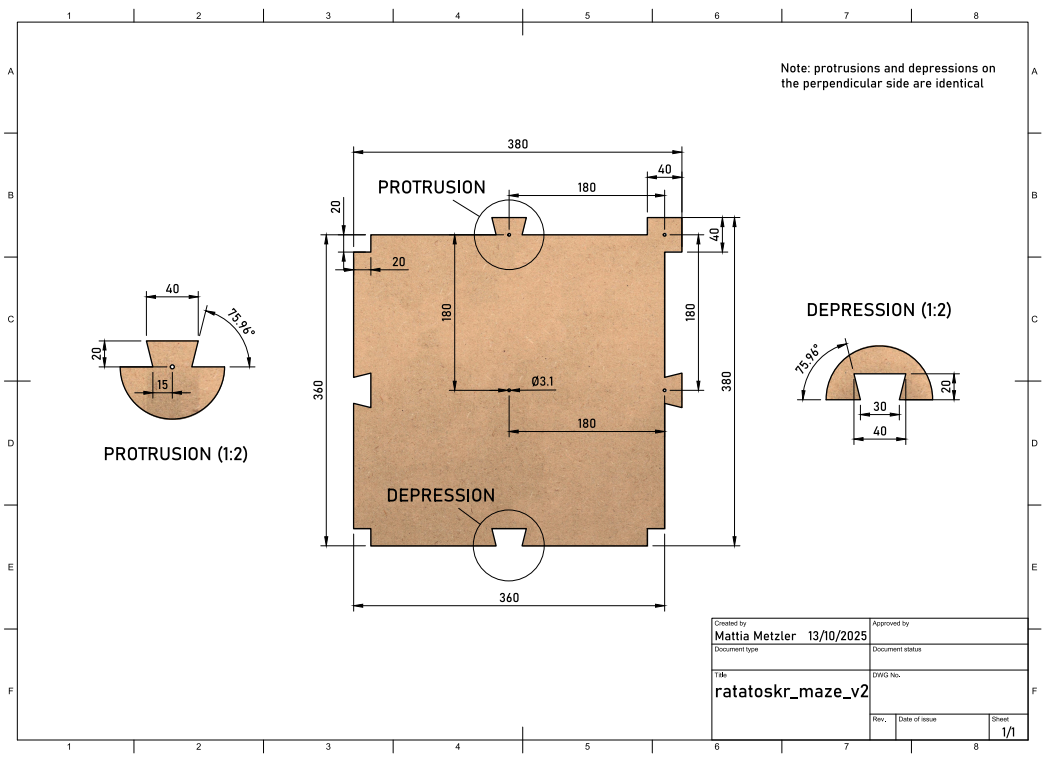


Figure 1: Technical drawing of the maze floor tile (created using Autodesk Fusion360)

3. The Mouse

This project builds upon last year's *MinosMouse* design, incorporating ideas from the *Picomouse* project and introducing several of our own improvements. Namely, one improvement is the ability to traverse diagonals in a completely straight line, as opposed to a zigzag motion. To accomplish this, we've decided on a combinations of motor encoders, sensors, and a gyroscope to accurately track the position and orientation of the mouse.

The Competition
Even though Ratatoskr will probably not compete in official MicroMouse competitions, we will still adhere to their rules, which are linked in full in Section 8.
We will also measure Ratatoskr's performance according to the official rules, which state:

1. Each mouse is allowed a maximum of 10 minutes to perform.
2. The scoring of a micro mouse shall be obtained by computing a handicapped time for each run as follows:

$$t_{\text{total}} = t_{\text{run}} + \frac{1}{30} \cdot t_{\text{search}} + p_{\text{touch}}$$

   where p_{touch} is a touch penalty and is calculated as $p_{\text{touch}} = 3 + \frac{1}{10} \cdot t_{\text{run}}$. It is only added if the mouse has been touched at any time between the search and the run.

These rules ensure that mice which use a lot of their maximal time finding the optimal path are penalized accordingly, as the goal of a good micromouse is to be not only fast but also efficient.

3.1. Design and Technical Drawing

Below is a technical drawing of the mouse, showing the design from multiple angles as well as some of its dimensions in millimeters. Note that these drawings don't yet include the newly conceptualized sensor layout as described in Section 3.2.3.

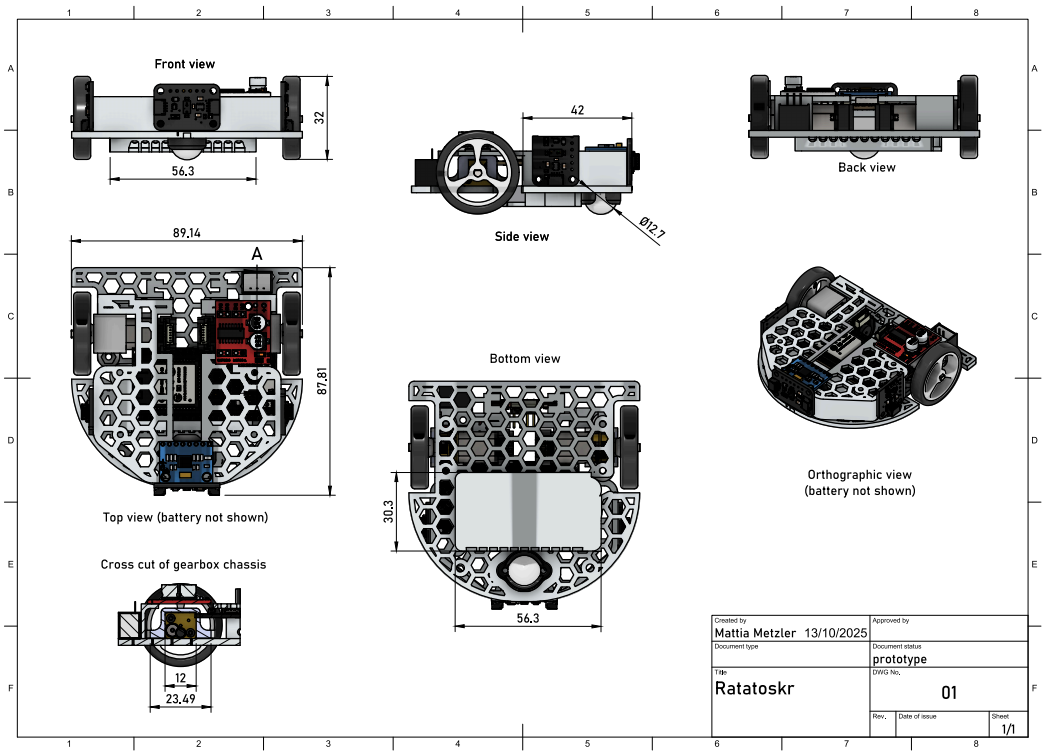


Figure 2: Technical drawing of Ratatoskr (created using Autedesk Fusion360)

3.2. Hardware

We will walk through the list of major hardware components used in Ratatoskr, explaining the reasoning and purpose of our choices.

3.2.1. Motors, Drivers & Encoders

Just like previous proven attempts, Ratatoskr will use two N20 DC gear motors with 750 RPM and built-in Hall encoders. We decided to keep this motor type since they've proven reliable and compact. The encoders allow for accurate position and speed control, which is necessary for both precise maze exploration and fast traversal.

The motors will be driven by an Adafruit DRV8871 DC Motor Driver Breakout Board (3.6 A max). We chose this driver since it offers a higher current limit and overall better efficiency, while remaining compact and easy to interface with. The DRV8871 provides reliable bidirectional control through standard PWM and direction pins, which makes it perfectly suited for our differential drive setup.

The ESP32 will generate PWM signals for each motor and handle encoder interrupts to read wheel rotation. This setup allows us to precisely regulate motor speed and direction while keeping the driver circuit simple and robust. The driver also includes built-in protection features such as current limiting, thermal shutdown, and undervoltage lockout, reducing the risk of hardware failure during testing.

We plan on using the encoders to estimate distance and direction by comparing the ticks of both wheels. This gives us an internal odometry system that can be cross-checked with the gyroscope and wall sensors later on.

For power, the motors run directly off the 7.4V LiPo battery[1] through the driver, while the rest of the system will be powered through a buck converter that brings the voltage down to 5V for logic and sensors. The encoders connect to interrupt-capable GPIO pins on the ESP32.

Overall, the hardware control of the motors will stay very similar to previous years' designs, but we will focus more on improving wheel control through better PID tuning and the combined use of encoder and gyro data to reduce slippage and drift. More importantly, we will aim to develop a combination of hardware and software for Ratatoskr to be able to traverse diagonals within the maze structure efficiently.

3.2.2. Gyroscope

For measuring orientation and rotational velocity, we are using an MPU-6050 IMU module. It's a 6-axis sensor that provides both accelerometer and gyroscope data over I^2C, and is commonly used in small robotic systems like ours.

The main purpose of the gyroscope is to keep track of Ratatoskr's direction and correct drift that occurs from wheel slip or uneven surfaces. During runs, the gyroscope will be used to ensure straight motion (keeping angular velocity close to zero) and to control turns precisely by integrating angular velocity until the desired rotation angle is reached.
The IMU connects via I^2C to the ESP32, and readings will be used with the wheel encoder data to improve reliability. At startup, the gyroscope will calibrate itself by measuring bias when the mouse is stationary. This zeroing process is necessary to prevent drift during longer runs.

We considered switching to a higher-end IMU (like the BMI160 used in some previous projects), but decided that the MPU-6050 is sufficient and easier to source locally. If we later notice

[1] Although initially, we will be using dry cell batteries for power

high drift or inconsistent readings, the module can be replaced without any major hardware or software changes, since the communication protocol remains the same.

3.2.3. TOF sensors

For wall detection and distance measurement, we are using four Time-of-Flight (ToF) sensors, similar to the setup used previously but slightly adjusted for our own needs. The sensors will help Ratatoskr detect nearby walls and align itself accurately within the maze.

We will use two VL53L4CD sensors facing forward to detect walls ahead of the mouse and to measure break distance at high speed. These long-range sensors allow the mouse to slow down early before hitting a wall and to plan turns smoothly.
Additionally, we will use two VL6180X sensors placed on the left and right sides of the mouse. These are short-range sensors used for centering between walls and for detecting open paths or intersections. The combination of both types gives us accurate coverage around the front half of the mouse.

All four sensors communicate over I^2C. Since they share the same default address, each one will be assigned a unique address during initialization using the XSHUT pins, or controlled through the I^2C multiplexer (see Section 7.)

In operation, the forward sensors will be used mainly for detecting walls and deceleration, while the side sensors will continuously poll and measure distances to maintain alignment. The software will filter these readings to ignore invalid values and handle possible occasional reflections from the maze which may mess with the sensors. This arrangement gives Ratatoskr a good field of view while avoiding the sensors overlapping and interfering with one another. See also Figure 4 for an overview of how these sensors will be used to navigate turns.

3.3. Software

The software for this project will be loosely based on the previous iterations that we have access to and therefore use `C++` as it was used in both projects and is generally the language of choice for such applications. We have decided on using the Google formatting style but using 4 spaces of indent, enforceable using the `.clang-format` file. All code that is not directly interacting with hardware should be testable without access to the mouse using unit tests and the mms micromouse simulation software.

3.3.1. Hardware Interfaces

We will need some software interfaces to communicate with the hardware mentioned earlier, the details of which are listed in the following.

3.3.1.1. Motor Interface Overview

The Adafruit DRV8871 DC Motor Driver Breakout Board provides two input pins (IN1 and IN2) that control the H-bridge and can accept PWM signals up to ~200kHz. The board supports 6.5–45V motor supply and up to 3.6A peak current, with built-in protections for overcurrent, thermal shutdown, and undervoltage lockout.

Attributes

`in1_pin`	first input pin connected to ESP32 PWM/logic GPIO
`in2_pin`	second input pin connected to ESP32 PWM/logic GPIO
`pwm_channel`	ESP32 PWM channel (if hardware PWM is used)
`max_pwm`	upper limit for PWM duty (0–255 or 0–1023 depending on mode)
`target_duty`	last commanded PWM duty cycle
`direction`	current direction (“forward”, “backward”, “brake”, “coast”)

Methods

`init()`	sets up both pins as outputs, attaches PWM channel, and ensures driver is idle.
`set_speed(duty)`	writes PWM value to selected input and LOW to the other pin.
`forward(speed)`	sets IN1 = LOW, PWM on IN2.
`backward(speed)`	sets IN2 = LOW, PWM on IN1.
`brake()`	drives both pins HIGH (active braking).
`coast()`	drives both pins LOW (free spin).
`stop()`	alias for `coast()`, used when idling or shutting down.
`test_ramp()`	optional helper to sweep PWM up/down for diagnostics.

Example usage

```
motorL = MotorDRV8871(in1_pin=25, in2_pin=26)
motorL.init()

# run forward half speed
motorL.forward(128)

# brake briefly, then reverse
motorL.brake()
delay(0.2)
motorL.backward(200)

motorL.stop()
```

All motor instances share the same pattern: one DRV8871 per wheel, each controlled through two GPIOs. Speed control is achieved purely through PWM duty cycle, and the higher-level motion control (PID, odometry, etc.) determines the duty target.

To conclude:

$$
\begin{aligned}
(\text{IN1} = \text{LOW}) \ \wedge \ (\text{IN2} = \text{PWM}) \ &\rightarrow \ \text{Forward} \\
(\text{IN1} = \text{PWM}) \ \wedge \ (\text{IN2} = \text{LOW}) \ &\rightarrow \ \text{Backward} \\
(\text{IN1} = \text{LOW}) \ \wedge \ (\text{IN2} = \text{LOW}) \ &\rightarrow \ \text{Coast} \\
(\text{IN1} = \text{HIGH}) \ \wedge \ (\text{IN2} = \text{HIGH}) \ &\rightarrow \ \text{Break}
\end{aligned}
$$

This layout gives us a clean, hardware-agnostic motor interface that remains valid even if the driver is swapped later (as long as it accepts PWM + DIR logic inputs).

3.3.1.2. ToF Interface Overview

Both ToF sensors speak I²C and measure distance via laser time-of-flight. The short-range VL6180X reliably covers up to ~200 mm which is ideal for side-wall centering, while the longer-range VL53L4CD covers up to ~1.2 m allowing forward look-ahead for fast runs. We will provide one abstract interface class in software that handles initialization, reading distance, error checking and supports multiple sensors. Sensors will be configured individually (side vs front) but use the same underlying driver logic. This gives us a clean way to plug whichever breakout we use (or future upgrade) and ensures the maze-solving logic can simply ask "what's my distance to wall X?" without worrying about the specifics.

All sensors share the same basic interface so they can be handled the same way, regardless of model or where they're mounted (front or side).

Attributes

`addr`	I²C address (unique per sensor after initialization)
`xshut_pin`	GPIO pin used to enable or disable the sensor
`role`	"front" or "side" (determines config like range and polling rate, can be enum)
`distance`	last valid distance reading (in mm)

`valid`	flag showing if the current reading is valid
`timestamp`	last time a valid reading was taken

Methods

`init()`	powers up the sensor, assigns I²C address, sets measurement mode and timing.
`start()`	begins continuous measurements or enables periodic polling.
`read()`	reads the latest distance value (returns distance in mm and a validity flag).
`stop()`	stops ranging and powers down the sensor (used when idling or resetting).
`reset()`	performs a soft reset if a sensor becomes unresponsive.
`is_valid()`	returns whether the current measurement is valid or timed out.
`filter()`	optional smoothing (median or moving average) to remove spurious spikes.

Example usage

```
tof_left = ToF(addr=0x2A, xshut_pin=17, role="side")
tof_left.init()
tof_left.start()

distance, valid = tof_left.read()
if valid:
  print("Left wall:", distance, "mm")
else:
  print("No valid reading")
tof_left.stop()
```

All ToF sensors will be managed through a shared driver that loops over each sensor object, polling and updating readings in the background. This lets higher-level modules (e.g., centering or wall detection) just call something like `get_wall_distance("left")` without worrying about the hardware details.

3.3.1.3. Gyroscope Interface Overview

The MPU-6050 sits on I²C (default addr `0x68`; `0x69` if `AD0` is high) and provides 3-axis gyro + 3-axis accel. We only require readings from this module so we will aim for the interface to be very simple.

Attributes

`addr`	I²C address (`0x68` or `0x69`)
`gyro`	latest gyro readings (x, y, z)
`accel`	latest accel readings (x, y, z)
`timestamp`	last time the sensor was updated
`valid`	flag indicating whether the last read was successful

Methods

`init()`	initializes I²C communication and wakes up the sensor.
`start()`	begins regular data polling from the IMU.
`read()`	returns the latest gyro and accel readings.
`stop()`	stops polling when the mouse is idle or powered down.
`is_valid()`	checks if the most recent data is valid.

Example Usage

```
imu = IMU_MPU6050(addr=0x68)
imu.init()
imu.start()

gyro, accel = imu.read()
if imu.is_valid():
    print("Gyro:", gyro, "Accel:", accel)
imu.stop()
```

This basic interface is enough for our needs: it lets us continuously read motion data and feed it into Ratatoskr's movement and orientation logic.

3.3.2. Movement Control

We will use a class to streamline the control of the mouse and to create an API which is able to interact with the simulator.

- `int mazeWidth()`
- `int mazeHeight()`
- `bool wallFront()`
- `bool wallRight()`
- `bool wallLeft()`
- `void moveForward(int distance = 1)`
- `void turnRight()`
- `void turnLeft()`
- `void setWall(int x, int y, char direction)`
- `void clearWall(int x, int y, char direction)`
- `void setColor(int x, int y, char color)`
- `void clearColor(int x, int y)`
- `void clearAllColor()`
- `void setText(int x, int y, String text)`
- `void clearText(int x, int y)`
- `void clearAllText()`
- `bool wasReset()`
- `void ackReset()`

3.3.3. PID Controllers (Proportional–integral–derivative)

To make Ratatoskr's movement precise and stable, we rely on PID control loops that constantly adjust the motor power based on sensor feedback. The idea is to let the mouse correct itself continuously on the fly.

In our implementation, we combine two kinds of control: one that regulates the wheel speeds through encoder feedback, and another that keeps the overall direction stable using the gyroscope. In practice, this means the encoders ensure both wheels spin at the correct rate, while the gyro makes sure the mouse maintains or changes its orientation exactly as commanded. Together they help Ratatoskr drive straight, turn cleanly, and recover quickly if one wheel slips.

The wheel speed PID adjusts the PWM sent to each motor so that the measured encoder speed matches the target speed. This keeps both wheels synchronized and allows for smooth acceleration. The PID in charge of direction, on the other hand, uses gyro and encoder data to correct deviations from the intended orientation. For instance, if the mouse starts veering slightly left or right while moving straight.

We will start with the PID parameters from MinosMouse as a reference, since they were tuned for a similar motor and weight setup. However, we'll re-tune them for Ratatoskr's specific mass distribution and the wooden maze surface. Tuning will be done empirically: testing response times, overshoot, and steady-state errors until the mouse can accurately complete physical test routes we will set up.

Each control loop runs at a fixed update rate (likely 500–1000 Hz), which is fast enough for stable control but not too heavy for the ESP32. The final PID equations will be implemented using integer arithmetic where possible to minimize floating-point overhead.

In short, the PID controllers ensure that Ratatoskr moves in a controlled, predictable way not too fast to slip, and not too slow to waste time. They also ensure that Ratatoskr will not crash

into walls at very high speeds due to the fact that PID uses real-time data and automatically adjusts the movement. Once properly tuned, they'll form the backbone of the motion control system.

3.3.4. Maze Solver

The mouse initially has no information on the maze which motivates the `search` traversal of the maze to discover the layout. As this phase is also timed and a fraction of it added to the final time, speed is not irrelevant. To save time we will employ an algorithm that increases the likelihood that the best path is inside of the explored labyrinth while not discovering every cell. The mouse needs to keep track of the walls present in each cell in order to construct a partial replica of the maze in its memory. We will approach exploration with a bias of moving towards the goal, where upon arriving the robot must make its way back to the start in order to initial the `run` phase. The way back to the start is used to discover more of the maze, once again using the `floodfill` algorithm. Using the micromouse simulator we can ensure that what the mouse wants to do is what it should do and optimize the path taken.

3.3.4.1. Diagonals

An additional issue for the solver will be the detection of diagonals. It should be possible to implement the detection of the type of diagonal and the encoding as post-processing steps that are applied to the path found by the simple solver. If time and resources allow, there is the possibility of refactoring the solver once the basic diagonals and straights work to search for the *fastest* instead of the *shortest* path. Further, the PID will need adjustments for the added complexity of sensor readings while traversing a diagonal.

3.3.5. LED Matrix & SD Card

As the battery on the micromouse is only sufficient for about half an hour of use, as observed by the MinosMouse team, we must make sure to waste as little battery life as possible. Thus, for debugging and logging we opt for a combined solution of an LED matrix and an SD card. This allows us to not be dependent on power-hungry systems like WiFi or Bluetooth.

The LED matrix will primarily be used during the runs, providing immediate feedback of the sensor data, for example which ToF sensors currently report a wall. A mockup of what this could look like can be seen in Figure 3.
The rest of the LEDs could be used to indicate other useful information, such as states of the state machine and so on.

The SD card will mainly be used after a run. It will collect and store information about the run at a much more granular level than the LED matrix could provide. Thus in combination, if a run has weird behavior, we can immediately check the LED matrix and then look at the debug data afterward.

The hardware-software interfaces for the LED matrix and SD card are quite simple, as they both use I^2C for communication and there are well documented libraries for both, namely the *Adafruit LED Backpack Library* and the builtin *SD* library.

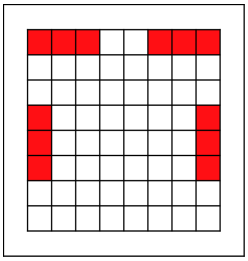
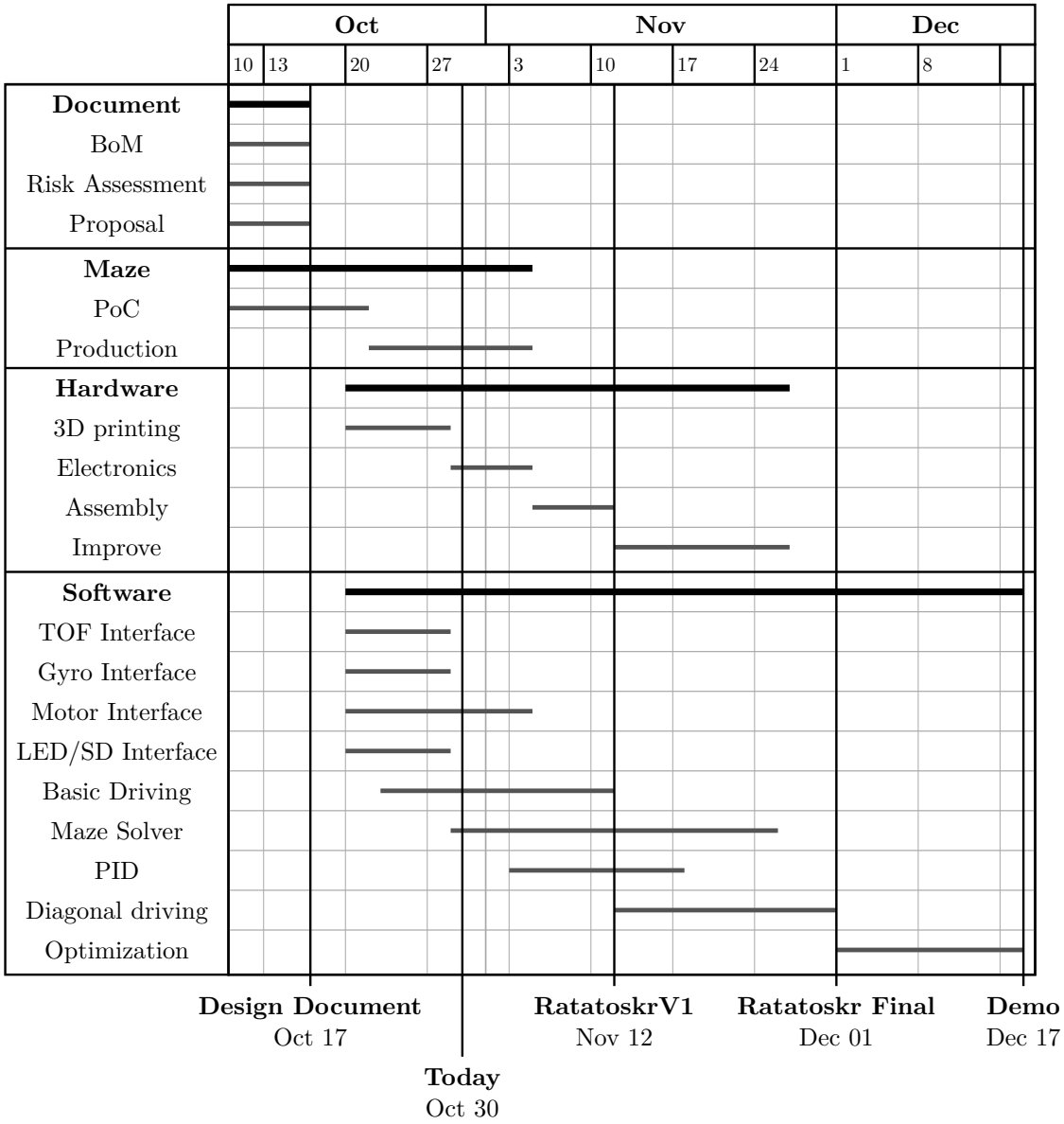


Figure 3: The LED matrix showing that all four sensors are currently sensing a wall

4. Tasks and Milestones

We have created a GANTT chart to provide an overview of the tasks to be done as well as their relations to the milestones and to each other.



4.1. Ratatoskr V1.0

The first version of the micromouse must be able to successfully maneuver and solve the maze in however much time it takes. This requires correct assembly and wiring of the physical components, working drivers for motors, gyroscope and sensors, a PID and an algorithm that can explore and solve the maze. At this points there is no need to worry about fancy turns or diagonals, a simple `stop - turn 90° - go` movement pattern will suffice, see type 0 in Figure 4.

4.2. Ratatoskr V1.1

To reach this milestone it is necessary to improve upon the first version by compressing successive forward movements into a single long straight. Further, the mouse should now be able to complete both type 1 and type 2 turns (Figure 4).

4.3. Ratatoskr V1.2

The next step will be to tackle diagonals. Requiring adjustments in the PID, two (possibly five) new turn types in order to efficiently and precisely handle entering, traversal and exiting. The algorithm composing the path for the final run will have to be adjusted to recognize diagonals and give the correct instructions.

4.4. Ratatoskr Final

The final version of our micromouse should be able to solve the maze by exploring it and then compute and execute a final run in which it never comes to a full stop, accelerates on long straights and speeds along diagonals where applicable.

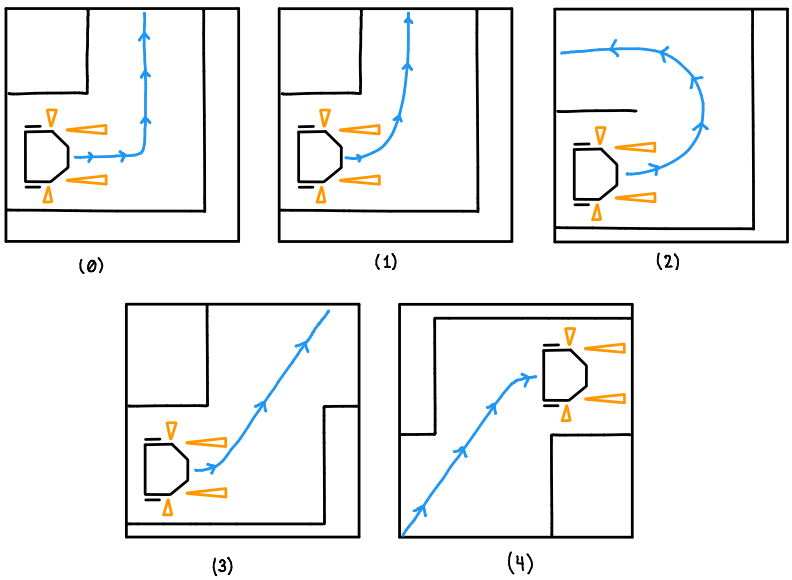


Figure 4: Turn Types

5. User Stories

As **a member of the team** working on Ratatoskr, I want the micromouse to be easy to service during development, for example by providing debug data after a run. Further, I want the finished micromouse and maze to be easy and moderately quick to set up and store, because I may want to show it off to friends.

More generally, as **a person interested in making things** I want to see that it is possible to create something fun and interesting, even with a limited background/knowledge in electronics and embedded programming.

As **the professor** I'd like parts of the project (or the project as a whole) to be reusable for future group or individual projects. Thus, I want the code and the design to be well-documented and modularized.

As **a viewer of a micromouse competition** I want the micromouse to perform well and be well-designed and compact. Optionally, it would be nice if the micromouse looked good, for example by incorporating some cosmetic aspects such as the LED matrix.

6. Risk Assessment

Regarding Ratatoskr, we will discuss the 3 elementary sources of issues and the risks they convey, namely hardware, software, and human resources. To measure the likelihood of these risks, we will use a scale of 1, meaning very unlikely, to 5, meaning very likely.

6.1. Hardware Risks

We made a list of a couple risks we will be facing in terms of hardware ordered from most likely to least likely but also with regard to importance:

- **Complexity of circuitry**
  - Likelihood: 4
  - Impact: 5
  - Effect: Use of multiple electrical components with all associated risks that come with it. Will lead to burning/ruining components.
  - Origin: Complexity will cause confusion on a hardware-level. This creates the risk
  - Prevention: Keeping the circuitry as simple and modular as possible. Sanity checking the design with multiple people and staff before implementing.
- **Insufficient power from motors (or hardware mismatch)**
  - Likelihood: 3
  - Impact: 4
  - Effect: Slower than expected mouse. One component can also become the weakest link, hindering other component capabilities (if motors are slow, then no use in getting top of the market gyros if our speed isn't even fast enough)
  - Origin: Miscalculation in design. Lack of foresight.
  - Prevention: Go over the design and plan multiple times with different people. Make sure to think until the very end of the assembly process.
- **Slippage on the new maze design**
  - Likelihood: 2
  - Impact: 3
  - Effect: Ratatoskr will not be able to make sharp turns. Slippage leads to uncertainty.
  - Origin: Trying out a new design for the maze floor which differs from the previous year's felted design.
  - Prevention: We believe MDF provides sufficient friction for Ratatoskr's tires. This belief, however, could be misplaced. In the worst-case scenario, we may use some rubberized coating seal, like Flex Seal or Plasti Dip.

6.2. Software Risks

There is also a list of software concerns we may likely face:

- **I2C/Communication issues with hardware**
  - Likelihood: 5
  - Impact: 3
  - Effect: If software does not correctly communicate with hardware, we may face endless amounts of debugging and researching ways to bypass some restrictions left by I2C and our implementation.

  - Origin: Lack of understanding or lack of infrastructure to seamlessly create an interface between the hardware we order and the software we make.
  - Prevention: Ensure to have completely read through documentation on the components along with their interfaces, specifically how they connect to software. If a component seems to cause problems, we can either rethink the component choice or think of a workaround.

- **Complexity of algorithms (algorithm mismatch)**
  - Likelihood: 3
  - Impact: 2
  - Effect: Nonoptimal attempts to solve/traverse the maze. Failed runs.
  - Origin: Lack of understanding regarding the complexity of an algorithm or how it will be implemented. Perhaps choice of algorithm may not suit specific types of mazes, issues with scaling to bigger mazes.
  - Prevention: We must keep an eye on the choice/complexity of the algorithm we will use for solving and traversing the maze in the shortest distance and fastest time. We should think of the different scenarios Ratatoskr may be put it (small mazes with an emphasis and speed or large mazes with an emphasis of time)

6.3. Human Resource Risks

Finally, we have a list of human-related issues and risk we will likely face.

- **Bugs encountered when developing**
  - Likelihood: 4
  - Impact: 3
  - Effect: The scale and complexity of the project may cause issues for the developing team. Malfunctioning code could be detrimental to Ratatoskr in some cases like crashing into maze walls.
  - Origin: Very likely but also very dependant on the team working on the project. It can be caused by either one person working on the code in its entirety and getting caught up by the complexity and scale, or it can due to lack of communication between the team members working on development. To add to this, debugging may be difficult with an autonomous piece of hardware. Live checks are not possible, thus hindering the debugging process.
  - Prevention: Through modularly choosing sections and tasks for development, we can spread the work through multiple people who master their own implementation and can effectively communicate their code with others. Having multiple people on the code may also help with frequent sanity checks on code logic. Logging data when completing runs may also help tremendously with the debugging procedure.

- **Mishandling of components**
  - Likelihood: 2
  - Impact: 3
  - Effect: May result in the destruction of the components we work with rendering them useless. That would in turn cause delays in the progress of the project (buying new components, figuring out fixes)
  - Origin: Lack of experience regarding the components we are dealing with. Lack of supervision when working with unfamiliar components.
  - Prevention: Ensuring to check with more experienced people like the staff when it comes to unfamiliar hardware.

7. Bill of Materials

Links and prices as of 19.10.2025.

Part	**Quantity**	**Price (CHF)**
N20 DC Gear Motors 750RPM *	2	31.92
MX1508 motor driver *	1	-
ESP32 development kit (30pin)	1	10.95
LM2596 Stepdown Buck Converter	1	3.71
VL53L1X (TOF400C) ToF sensor (set of 4) *	1	16.66
MPU-6050 3-axis accelerometer with gyroscope	1	3.95
8x8 LED-Matrix w/ I2C Backpack	1	11.90
Adafruit MicroSD card breakout board	1	12.90
18/27mm tires for RC cars (pair)	2	5.00
9V battery + connector *	1	-
Total	-	**96.99**

* components in stock for the course, as discussed
We need four tires total, as we might want to have four wheels on the mouse.

8. Inspiration and References

Last year's micromice:
- MinosMouse: https://github.com/epfl-cs358/2024fa-minosmouse
- PicoMouse: https://github.com/epfl-cs358/2024fa-picomouse

Official micromouse rules: https://micromouseonline.com/micromouse-book/rules/